

FCS Endsem Exam

Name: Utkarsh Arora

Roll No: 2020143

Q1a) DNS poisoning, also known as DNS spoofing, is a type of cyber-attack in which an attacker takes advantage of vulnerabilities in the domain name system (DNS) to redirect internet traffic away from legitimate servers and towards malicious ones. This can be done by corrupting the DNS server's records, so that when a user tries to access a legitimate website, they are instead directed to a fake website that is under the attacker's control. This type of attack can be used to steal sensitive information, such as login credentials, or to spread malware to unsuspecting users.

We can see that the Attacker is on the same local network as Alice. Thus, the DNS poisoning attack can be carried out locally.

The Local DNS server is present in the Local Network. Thus, we have two main options to perform the attack.

In the first method, the attacker can try to gain access to the Local DNS Server. These depend on the security of the local DNS server. There are various ways to gain access to a server, all depending on how secure the server is. First the attacker could try accessing the servers physically if he has the access to the same. He could plug in a system to the server to gain access. Also, the attacker could try exploiting vulnerabilities in the server's software. Known vulnerabilities in a server's operating system or applications can be used to gain access to the system without needing to guess login credentials.

The attacker could also try using login credentials. Many servers come with default login credentials that are well-known, which the attacker could try. He may also be able to guess login credentials based on publicly available information about the server or its owner. The attacker could also try to use social engineering to trick users into revealing their login credentials, such as phishing attacks or impersonating a trusted authority. The attacker could also try to brute force the credentials, using automated tools to try a large number of username and password combinations in order to guess the correct ones.

Once the attacker has access to the system, he can make changes in the Local DNS Server. This would lead to DNS Poisoning, leading to Alice's DNS requests returning the wrong output.

The other method involves abusing the DNS Resolver. When a user sends a request, the DNS resolver looks up the IP address in its cache or send a query to a DNS server to obtain the correct IP address. The DNS resolver will then respond to the user's device with the correct IP address, allowing the device to connect to the website.

The attacker can perform DNS poisoning by **cache poisoning**. It is assumed that the DNS Resolver software is running on the switch.

- First, the attacker must gain access to the DNS Resolver. As the attacker is on the local network, this can be done by exploiting vulnerabilities in the DNS resolver's software or by using brute-force techniques to guess the resolver's login credentials. Again, this part can be done how it was done in the first method.
- Once the attacker has gained access to the DNS resolver, they can then send it forged DNS responses that contain false information about the mapping between domain names and IP addresses. This can cause the DNS resolver to update its cache with the false information.
- Hence, once Alice sends out a DNS request, the request will instead be routed to the servers the attacker has marked.
- The attacker can create fake websites, where confidential credentials have to be entered.
- Once Alice reaches that website, she would put in the credentials, and thus the attacker would have access to these credentials.

Q1b) A TCP SYN attack is a type of denial-of-service (DoS) attack in which an attacker sends a large number of SYN requests to a server, overloading it and preventing it from responding to legitimate requests. SYN requests are used in the TCP three-way handshake to initiate a connection between two devices, and a SYN attack exploits this process by sending a large number of SYN requests without completing the handshake.

In a TCP SYN attack, the attacker will typically use a network of compromised devices, known as a botnet, to generate a large number of SYN requests and send them to the target server. This can overwhelm the server, causing it to become unresponsive and preventing it from serving legitimate requests.

TCP SYN attacks can be difficult to defend against, as they can be launched from a large number of devices simultaneously. To counter TCP SYN attacks, there are several solutions that can be implemented, including the following:

1. A **SYN cookie mechanism** is a technique that can be used to protect against TCP SYN attacks. In a SYN cookie mechanism, the server will keep track of the initial sequence numbers (ISNs) that are generated in SYN requests. When it receives a SYN request, the server will generate a "cookie" that is derived from the ISN and other information about the request, and will send this cookie back to the client in the SYN-ACK response.

When the client receives the SYN-ACK response, it must include the cookie in its ACK message in order to complete the three-way handshake and establish a connection. This means that the server does not need to maintain any state information about the SYN request, as the cookie contains all of the information that is needed to complete the handshake.

The SYN cookie mechanism is effective because it allows the server to handle a large number of SYN requests without consuming any resources. Since the server does not need to maintain any state information about the SYN requests, it can handle a much higher number of requests without becoming overloaded. This makes it much more difficult for an attacker to launch a successful TCP SYN attack against the server.

2. Install a **SYN flood protection system** on the network. A SYN flood protection system is a type of network security measure that can help to protect against TCP SYN attacks. In a SYN flood attack, an attacker sends a large number of SYN requests to a server, overloading it and preventing it from responding to legitimate requests.

A SYN flood protection system can help to defend against these types of attacks by limiting the number of SYN requests that the server will accept from a single source. For example, the system may allow the server to accept a certain number of SYN requests from a single IP

address within a given time period. If the server receives more SYN requests than this limit, it will automatically drop any additional requests.

This type of system is effective because it can prevent the server from becoming overwhelmed by SYN requests. By limiting the number of SYN requests that the server will accept, the system can help to ensure that the server remains responsive and can continue to serve legitimate requests. This can make it much more difficult for an attacker to launch a successful SYN flood attack against the server.

3. Use a **load balancer** or **content delivery network** (CDN) to distribute incoming traffic across multiple servers. Load balancers and CDNs can help to defend against these types of attacks by distributing incoming traffic across multiple servers. This can help to prevent any single server from becoming overwhelmed by SYN requests, as the load balancer or CDN will distribute the requests among the available servers. This can help to ensure that the server remains responsive and can continue to serve legitimate requests, even in the face of a SYN flood attack. Load balancers and CDNs are effective in protecting against TCP SYN attacks because they can distribute incoming traffic among multiple servers, preventing any single server from becoming overloaded. This can help to ensure that the server remains responsive and can continue to serve legitimate requests, even in the face of a SYN flood attack.

Overall, these solutions can be effective in countering TCP SYN attacks because they allow the server or network to handle incoming SYN requests in a more efficient and secure manner, reducing the impact of the attack and protecting the server's connection queue from being exhausted.

Q1c) A DDoS (Distributed Denial of Service) attack is a type of cyber-attack in which an attacker attempts to make a network or server unavailable to its intended users. This is typically done by overwhelming the target with a flood of internet traffic, either by using a network of compromised computers (known as a botnet) or by using a distributed network of legitimate devices that have been recruited to participate in the attack (known as a DDoS amplification attack).

The goal of a DDoS attack is to prevent the targeted server or network from being able to serve legitimate requests, making it unavailable to its intended users. This can be done by consuming all of the available bandwidth, by overwhelming the server with requests, or by using other tactics to disrupt the server's ability to function.

In the situation provided to us, here is how attacker 2 can DDoS our website:

The given information is that the website is hosted on a webserver which is connected to the internet, and the attacker is also connected to the internet. Thus, from the given information, there is no one attack that appears to be the best attack in this case. Hence, I am going to describe multiple possible attacks that can be performed on the website, and the possible mitigations for them.

Broadly, there are three main types of DDoS attacks:

1. **Volume-based attacks:** These attacks involve overwhelming the target website with a large amount of internet traffic, typically by using a botnet to generate traffic from multiple sources simultaneously. This can consume all of the available bandwidth, preventing the website from serving legitimate requests.
2. **Protocol attacks:** These attacks exploit vulnerabilities in the protocols that are used to transmit internet traffic, such as TCP or UDP. By sending carefully crafted requests to the target website, the attacker can cause the server to use up all of its available resources, making it unable to serve legitimate requests.
3. **Application-layer attacks:** These attacks target the application layer of the website, where the server processes requests and sends responses. By sending a large number of requests that are designed to consume the server's resources, the attacker can cause the website to become unresponsive.

Volume based attacks

One example of a volume-based DDoS attack is a **UDP flood attack**. In this type of attack, the attacker sends a large number of UDP packets to random ports on the target system. Because UDP is a connectionless protocol, the target

system is unable to confirm that the packets are legitimate, and as a result, it must process them all. This can quickly overwhelm the system, making it unavailable to legitimate users.

Another example of a volume-based DDoS attack is an **ICMP flood attack**. In this type of attack, the attacker sends a large number of ICMP echo request (ping) packets to the target system. Because the target system must respond to each of these requests, it can quickly become overwhelmed, making it unavailable to legitimate users.

A third example of a volume-based DDoS attack is a **TCP SYN flood attack**. In this type of attack, the attacker sends a large number of TCP SYN packets to the target system, but never completes the connection. This can cause the target system to become overwhelmed with half-open connections, making it unable to process legitimate requests.

As a website developer, there are several measures you can take to prevent volume-based DDoS attacks.

One effective way to protect against these attacks is to use a **web application firewall (WAF)**. A WAF is a security tool that sits between your website and the internet, and it analyzes incoming traffic in real-time to identify and block malicious traffic. By using a WAF, you can prevent most volume-based DDoS attacks.

Another way to protect against volume-based DDoS attacks is to use a **content delivery network (CDN)**. A CDN is a network of servers that are distributed across the internet, and it can be used to distribute the load of incoming traffic across multiple servers. This can help prevent a single server from becoming overwhelmed by a large volume of traffic, and it can make your website more resilient to DDoS attacks.

Additionally, you can use **rate limiting** to prevent DDoS attacks. Rate limiting is a technique that limits the number of requests that a user can make to your website within a specific time period. By implementing rate limiting, you can prevent attackers from overwhelming your website with a large volume of traffic, as they will be unable to make enough requests to cause a disruption.

Overall, the best way to protect against volume-based DDoS attacks is to implement a combination of these measures, as well as to regularly monitor

your website for signs of an attack and to have a response plan in place in case an attack does occur.

Protocol Based Attacks

A protocol-based DDoS attack is a type of cyber-attack that targets the underlying communication protocols used by a website or network. These attacks aim to exploit vulnerabilities in these protocols in order to overwhelm the target system and make it unavailable to users. Some common types of protocol-based attacks include SYN floods, fragmented packet attacks, and ping of death attacks. These attacks can be difficult to defend against, as they often involve carefully crafted packets that can bypass many security measures.

One example of a protocol-based DDoS attack is a **SYN flood attack**. In this type of attack, the attacker sends a large number of SYN packets to the target system, but never completes the TCP connection. This can cause the target system to become overwhelmed with half-open connections, making it unable to process legitimate requests.

Another example of a protocol-based DDoS attack is a **fragmented packet attack**. In this type of attack, the attacker sends a large number of packets that have been fragmented into multiple smaller packets. Because the target system must reassemble these packets, it can quickly become overwhelmed, making it unavailable to legitimate users.

A third example of a protocol-based DDoS attack is a **ping of death attack**. In this type of attack, the attacker sends a series of ping packets that are larger than the maximum allowed size. When the target system receives these oversized packets, it can crash or become unresponsive, making it unavailable to legitimate users.

As a website developer, there are several measures you can take to prevent protocol-based DDoS attacks.

One effective way to protect against these attacks is again to use a **web application firewall (WAF)**. WAF prevents against Protocol Based DOS attacks like it prevents against Volume Based attacks.

Another way to protect against protocol-based DDoS attacks is to keep your website and network infrastructure up-to-date with the latest security patches and updates. Protocol-based attacks often exploit vulnerabilities in outdated software, so by keeping your website and network infrastructure up-to-date, you can reduce the likelihood of an attack.

Additionally, you can use **network segmentation** to prevent DDoS attacks. Network segmentation is the practice of dividing your network into smaller, isolated segments, which can help prevent attackers from gaining access to your entire network. By implementing network segmentation, you can limit the potential damage of a DDoS attack and make it easier to identify and isolate the source of the attack.

Overall, the best way to protect against protocol-based DDoS attacks is to implement a combination of these measures, as well as to regularly monitor your website for signs of an attack and to have a response plan in place in case an attack does occur

Application-layer Based Attacks

An application-layer DDoS attack is a type of cyber-attack that targets the application layer of the OSI model. This is the layer of the network where most user-facing applications operate, such as web servers, email servers, and database servers. These attacks aim to exploit vulnerabilities in these applications in order to overwhelm the target system and make it unavailable to users. Some common types of application-layer attacks include HTTP flood attacks, slowloris attacks, and SQL injection attacks. These attacks can be difficult to defend against, as they often involve manipulating legitimate requests in order to overwhelm the target system.

One example of an application-layer DDoS attack is an **HTTP flood attack**. In this type of attack, the attacker sends a large number of HTTP requests to the target web server, with the goal of overwhelming the server and making it unavailable to legitimate users.

Another example of an application-layer DDoS attack is a **slowloris attack**. In this type of attack, the attacker sends a large number of incomplete HTTP requests to the target web server. Because the requests are never completed,

the server is forced to keep these connections open, eventually running out of resources and becoming unavailable to legitimate users.

A third example of an application-layer DDoS attack is a **SQL injection attack**. In this type of attack, the attacker sends malicious SQL commands to a target database server, with the goal of overwhelming the server and making it unavailable to legitimate users.

As a website developer, there are several measures you can take to prevent application-layer DDoS attacks.

One effective way to protect against these attacks is again to use a **web application firewall (WAF)**. WAF is useful in preventing a lot of types of DOS attacks.

Another way to protect against application-layer DDoS attacks is to implement **input validation** on your website. Input validation is the process of ensuring that all user-supplied data is valid and safe before it is processed by your application. By implementing input validation, you can prevent attackers from injecting malicious data into your application, which can help protect against SQL injection attacks and other types of application-layer DDoS attacks.

Additionally, you can use **network segmentation** to prevent DDoS attacks. Network segmentation is the practice of dividing your network into smaller, isolated segments, which can help prevent attackers from gaining access to your entire network. By implementing network segmentation, you can limit the potential damage of a DDoS attack and make it easier to identify and isolate the source of the attack.

Overall, the best way to protect against application-layer DDoS attacks is to implement a combination of these measures, as well as to regularly monitor your website for signs of an attack and to have a response plan in place in case an attack does occur.

If access to your website has been disrupted by a DDoS attack, there are several steps you can take to restore access.

The first step is to identify the source of the attack and block the traffic. This can be done by analyzing your website's logs and traffic data to identify the IP addresses that are sending the malicious traffic. Once you have identified the source of the attack, you can block this traffic at the network level, either by updating your firewall rules or by contacting your hosting provider and asking them to block the traffic.

The next step is to implement additional security measures to prevent the attack from happening again. This may involve using a web application firewall (WAF) to block malicious traffic, implementing rate limiting to prevent attackers from overwhelming your website with traffic, or using network segmentation to isolate your website from the rest of your network.

Once you have implemented these measures, you can begin to restore access to your website. This may involve restoring your website from a backup, if the attack caused any data loss, or it may involve cleaning up any damage that was done to your website's files or database.

Once your website is back online and accessible to legitimate users, you should continue to monitor it for signs of ongoing attacks or other security issues. It is also important to have a response plan in place in case the attack happens again, so that you can quickly restore access to your website and prevent any further disruption.

Q2a) An SSL strip attack is a type of cyber-attack that targets the HTTPS protocol, which is used to encrypt communications between a website and its users. In an SSL strip attack, the attacker intercepts and modifies the traffic between the user and the website, stripping away the SSL encryption and replacing it with unencrypted HTTP. This allows the attacker to view and modify the traffic, potentially stealing sensitive information such as login credentials or credit card numbers. The goal of an SSL strip attack is to trick the user into thinking that their connection to the website is secure, when in fact it is not.

This is a Man-In-The-Middle attack.

To perform this attack:

- First, we would have to intercept the traffic between the website and the user.
- This can be done by setting up a rogue access point or through **ARP spoofing**. ARP spoofing, also known as ARP poisoning or ARP cache poisoning, is a type of cyber-attack in which an attacker sends falsified ARP (Address Resolution Protocol) messages over a local area network (LAN). This is done in order to associate the attacker's MAC address with the IP address of another device on the network, such as a router or a server. When a device on the network wants to communicate with the IP address of the router or server, it will send the data to the attacker's MAC address instead, because the ARP cache on the device has been poisoned with the attacker's MAC address.
- The IP Tables have to be modified to set that any connection coming via port 80 has to be routed through port 10000. This is because sslstrip uses port 10000 by default.
- We figure out the interface on which this communication is running.
- Then, we run the **sslstrip script**. Sslstrip is a python script. The script can be found here: <https://github.com/moxie0/sslstrip>. We must install python before running the script.

The SSLStrip Man-In-The-Middle attack is complete. When accessing the website, the user will be led to an http version of the website instead of the https version. Any credential they enter will be unencrypted. As we have already set up a Man-In-The-Middle listening node, we can listen to this node and get this unencrypted private data.

Q2b) These attacks can be prevented both on the client-side and the server-side.

On the client side;

1. Use **HTTPS Everywhere**: HTTPS Everywhere is a browser extension that automatically enables HTTPS on websites that support it. This can prevent SSLstrip attacks by ensuring that your connection to the

website is always encrypted, even if the attacker tries to strip the HTTPS protocol.

2. Install a **certificate pinning plugin**: Certificate pinning is a security measure that prevents SSLstrip attacks by requiring the web browser to only trust a specific set of certificates for a particular website. By installing a certificate pinning plugin, you can prevent SSLstrip attacks by ensuring that your browser only trusts the correct certificates for the websites you visit.
3. Use a **VPN**: A virtual private network (VPN) encrypts all of your internet traffic and routes it through a secure server. This can prevent SSLstrip attacks by encrypting your connection to the website, even if the attacker tries to strip the HTTPS protocol.

On the server side,

1. Use HSTS: **HTTP Strict Transport Security (HSTS)** is a security measure that forces the client (e.g. a web browser) to only communicate with a server using HTTPS. This can prevent SSLstrip attacks by ensuring that the client always uses HTTPS to connect to the server, even if the attacker tries to strip the HTTPS protocol.
2. Use **certificate transparency**: Certificate transparency is a security measure that requires all SSL certificates to be logged and publicly available. This can help to detect SSLstrip attacks by allowing anyone to verify that a certificate is valid and has not been tampered with.
3. Use a **content security policy**: A content security policy (CSP) is a security measure that allows a server to specify which external resources (e.g. scripts, stylesheets, images) are allowed to be loaded on a page. By using a CSP, a server can prevent SSLstrip attacks by only allowing HTTPS resources to be loaded on the page, which ensures that the connection to those resources is always encrypted.

The effectiveness of the measures to counter a SSLstrip man-in-the-middle attack will vary depending on the specific circumstances and the specific measures being used. However, in general, these measures can be effective in protecting against SSLstrip attacks and other types of man-in-the-middle attacks.

For example, using HTTPS Everywhere can be very effective in preventing SSLstrip attacks, as it ensures that the connection to the website is always encrypted. Similarly, using a certificate pinning plugin can also be effective, as it prevents the attacker from being able to use a forged certificate to impersonate the website. Using a VPN can also be effective in protecting against SSLstrip attacks, as it encrypts all of your internet traffic and routes it through a secure server. However, the effectiveness of a VPN will depend on the specific VPN provider and the VPN protocol being used.

On the server-side, using HSTS can be effective in preventing SSLstrip attacks, as it forces the client to use HTTPS to connect to the server. Similarly, using certificate transparency and a content security policy can also be effective in detecting and preventing SSLstrip attacks, as they allow the server to verify the validity of SSL certificates and control which resources are allowed to be loaded on the page.

Overall, these measures can be effective in protecting against SSLstrip attacks, but it is important to use a combination of measures and to regularly update and maintain them to ensure their continued effectiveness.

Q3a) As the chief information security officer (CISO) for a large financial institution, my top priority would be to ensure the security of our company's IT systems and data during the merger process with another financial institution. To do this, I would take the following steps:

- Conduct a thorough **risk assessment** to identify any potential vulnerabilities or threats that may arise as a result of the merger. This could involve analyzing the existing IT infrastructure and security protocols of both companies, as well as any potential gaps or overlaps in coverage. I would also consider any external factors that could affect the security of our systems, such as the threat landscape and the regulatory environment.
- Develop a **risk management plan** that outlines the steps that will be taken to mitigate or eliminate the identified risks. This plan could include measures such as implementing additional security controls, conducting regular security assessments and audits, and establishing protocols for responding to security incidents. I would also consider the costs and

benefits of each proposed measure, to ensure that we are taking a cost-effective approach to managing risks.

- Communicate the risk management plan to the other executives at the company, as well as to any relevant stakeholders who will be involved in the merger process. This could include presenting the plan at a meeting, sending out a memo or email, or providing written documentation that outlines the key components of the plan. I would make sure to clearly explain the potential risks and the steps that will be taken to manage them, to ensure that everyone understands the importance of the plan and their role in implementing it.
- Monitor and update the risk management plan throughout the integration process, to ensure that it remains effective and relevant as the merger progresses. This could involve conducting regular reviews and assessments to identify any new risks that may arise, and adjusting the plan accordingly. I would also make sure to regularly communicate any updates or changes to the plan to the relevant stakeholders, to ensure that everyone is aware of the current state of our risk management efforts.

Overall, my approach to managing the security risks associated with integrating the IT systems of our company and another financial institution would involve conducting a thorough risk assessment, developing a comprehensive risk management plan, and communicating that plan to relevant stakeholders. By taking these steps, I would aim to ensure that our company is prepared to deal with any potential security threats and vulnerabilities that may arise during the merger process, and to protect the sensitive data and systems that are critical to our operations.

Q3b) We use the following formula to calculate the Risk Value:

$\text{Risk value} = \text{likelihood} * \text{value} * (1 - \text{control} + \text{assumption accuracy})$

For Vulnerability #1:

Likelihood = 0.4

Value = 50

Control = 0

Assumption Accuracy = 0.3

For Vulnerability #2:

Likelihood = 0.3

Value = 40

Control = 0.5

Assumption Accuracy = 0.2

For Vulnerability #3:

Likelihood = 0.5

Value = 40

Control = 0.2

Assumption Accuracy = 0.2

For information asset A, the risk value is: $0.4 * 50 * (1 - 0 + 0.3) = \mathbf{26}$

For information asset B, the risk value is: $(0.3 * 40 * (1 - 0.5 + 0.2)) + (0.5 * 40 * (1 - 0.2 + 0.2)) = \mathbf{28.4}$

Clearly, **asset B has a higher risk value.**

Therefore, asset B is more important to the protection of the organization's information.

Q4a)

I would take the following actions to protect other users if I suspected that a user's account had been hacked by a CSRF (cross-site request forgery) attack:

- I would first make sure that a CSRF attack has indeed compromised the user's account. This can be achieved either by scanning the website's logs for any unusual behaviour or by looking for indications of unauthorised activity, such as the user logging in from a different device or location than normal.
- I would immediately invalidate the user's current session and make them log in again after I had verified that the user's account had been compromised. This will prevent the attacker from continuing to access the user's account.
- Next, I would implement measures to prevent future CSRF attacks on the website. This can be done by implementing CSRF tokens, which are unique, randomly generated tokens that are attached to each form submission on the website. These tokens are checked on the server-side to ensure that the request is coming from a legitimate source.
- In addition to implementing CSRF tokens, I would also recommend implementing other security measures, such as rate limiting and captcha verification, to further protect the website from attack.
- Finally, I would alert other users of the potential security threat and advise them to take steps to protect their own accounts, such as using strong, unique passwords and enabling two-factor authentication.

Q4b) If I suspected that my website had been the target of an XSS (cross-site scripting) attack, I would take the following steps to prevent it from happening again and to clean up any malicious code that may have been injected into the site:

- First, I would verify that the website was indeed the target of an XSS attack. This can be done by looking for signs of malicious code in the product descriptions, such as unusual JavaScript or HTML code, or by inspecting the website's logs for any suspicious activity.
- Once I confirmed that the website was the target of an XSS attack, I would immediately remove any malicious code that may have been injected into the site. This may involve manually searching through the website's source code and removing any code that is not necessary for the website's functionality.

- Next, I would implement measures to prevent future XSS attacks on the website. This can be done by implementing input validation and sanitization, which involves checking all user-supplied input for potentially dangerous characters or code and removing or encoding them as necessary.
- In addition to implementing input validation and sanitization, I would also recommend implementing other security measures, such as content security policies and HTTP headers, to further protect the website from attack.
- Finally, I would alert other users of the potential security threat and advise them to take steps to protect themselves, such as not clicking on suspicious links or downloading unknown files.

Q4c) If I suspected that my website had been the target of a SQL injection attack, I would take the following steps to prevent it from happening again and to secure my database against future attacks:

- First, I would verify that the website was indeed the target of a SQL injection attack. This can be done by looking for signs of SQL injection in the website's logs, such as unusual SQL queries or error messages, or by inspecting the database for any suspicious activity.
- Once I confirmed that the website was the target of a SQL injection attack, I would immediately take the website offline to prevent further damage. This will also give me time to clean up any malicious code or data that may have been injected into the database.
- Next, I would implement measures to prevent future SQL injection attacks on the website. This can be done by implementing input validation and sanitization, which involves checking all user-supplied input for potentially dangerous characters or code and removing or encoding them as necessary.
- In addition to implementing input validation and sanitization, I would also recommend implementing other security measures, such as prepared statements and stored procedures, to further protect the database from attack.

- Finally, I would alert other users of the potential security threat and advise them to take steps to protect their personal information, such as not providing their sensitive information to unknown websites or individuals.

Q5a) Buffer overflow attacks are a type of vulnerability that can occur in software programs when a program tries to store more data in a buffer (a temporary storage area in memory) than it was designed to hold. This can cause the program to crash or, in the case of a malicious attacker, allow the attacker to execute arbitrary code on the target system.

Secure Sockets Layer (SSL) is a cryptographic protocol that provides secure communication over a computer network. It is commonly used to encrypt data sent over the internet, such as when using a web browser to access a secure website. However, like any software, the SSL protocol and the libraries used to implement it can be vulnerable to buffer overflow attacks.

It is possible for an attacker to exploit a buffer overflow vulnerability in Secure Sockets Layer (SSL) by executing malicious code on a targeted system. In the case of the OpenSSL library, I could execute a buffer overflow attack by sending a large amount of data to a vulnerable OpenSSL server, causing it to crash or behave unexpectedly.

To execute such an attack, I would first need to identify a vulnerable OpenSSL server. I can do this by scanning for servers that are running older versions of OpenSSL, which are more likely to be vulnerable to buffer overflow attacks.

Once I have identified a vulnerable server, I will craft a malicious payload that contains code designed to exploit the buffer overflow vulnerability. This payload can be sent to the server in the form of a large amount of data, such as a long string of characters or a large file.

I could send a large amount of data to the server using tools such as the "openssl" command-line utility. This data could be specifically designed to exploit the vulnerability in the OpenSSL library, causing the buffer overflow and allowing the attacker to execute arbitrary code on the server.

Alternatively, I could use a network scanner to identify systems that are using the vulnerable version of the OpenSSL library, and then use a pre-existing exploit to attack those systems.

If the server is vulnerable, the malicious payload will cause it to crash or behave unexpectedly, allowing me to potentially gain unauthorized access to the server or execute arbitrary code on it.

Q5b) To defend against buffer overflow attacks on SSL, it is important to take steps to prevent the attacks from occurring in the first place. This can be done by implementing the following measures:

- Keep all software and libraries, such as the OpenSSL library, up to date with the latest security patches and updates. This will help to protect against known vulnerabilities and reduce the likelihood of a successful buffer overflow attack.
- Additionally, using a network intrusion detection system (NIDS) or a web application firewall (WAF) can help to detect and block malicious traffic that may be attempting to exploit vulnerabilities in SSL.
- Implement input validation and sanitization to ensure that all user-supplied input is checked for potentially dangerous characters or code. This will help to prevent attackers from injecting malicious payloads into the server.
- One important measure is to regularly monitor and assess the security of the systems, including conducting regular penetration tests to identify any vulnerabilities that may be present. This can help organizations to detect and fix vulnerabilities before they can be exploited by attackers.
- Organizations can also implement security policies and procedures that specify the steps that should be taken to prevent, detect, and respond to security incidents. This can include measures such as implementing access controls to limit who has access to sensitive data and systems, regularly backing up data to prevent loss in the event of an attack, and establishing incident response protocols to ensure that security incidents are quickly and effectively dealt with.

Ultimately, defending against buffer overflow attacks on SSL-enabled systems requires a comprehensive approach that addresses both the technical and the human aspects of security. By implementing the measures discussed above, organizations can significantly reduce the risk of their systems being exploited by attackers and protect the sensitive data that they transmit over the internet.

Bonus)

Zombinder is a service that allows users to bind malicious payloads, such as droppers, to legitimate, everyday apps. This allows attackers to deliver malware to targets in a way that is difficult to detect, as the malicious app may appear to be a legitimate app.

A dropper is a type of malware that is used to deliver other malware to a target. Droppers are often used by attackers to gain access to a device or network and to install additional malware, such as ransomware or spyware. Droppers are typically hidden within legitimate-looking apps or files and are executed when the app or file is opened.

One potential attack that uses Zombinder (or similar services) to bind dropper capabilities to everyday apps could involve the attacker creating a malicious app that appears to be a legitimate, popular app, such as a game or social media platform. The attacker could then use Zombinder to bind the dropper capabilities to the app, allowing the attacker to execute arbitrary code on the victim's device when the app is installed and run.

To create the malicious app, the attacker would first need to have knowledge of app development and the ability to create a convincing replica of a popular app. The attacker could then use Zombinder to bind the dropper capabilities to the app, which would allow the attacker to execute arbitrary code on the victim's device when the app is installed and run.

Once the malicious app is ready, the attacker could distribute it through various channels, such as social media or app stores. The attacker could use various tactics to trick victims into downloading and installing the app, such as creating fake user reviews or using social engineering techniques to convince victims that the app is legitimate.

Once the app is installed and run on the victim's device, the dropper that was bound to the app using Zombinder would execute and install the backdoor, giving the attacker remote access to the device. The attacker could then use this access to steal sensitive information from the device, such as login credentials or financial information.

This type of attack can be difficult to detect, as the malicious app may appear to be a legitimate app and the dropper may not be immediately visible to the user. To defend against this type of attack, it is important to use caution when downloading and installing apps, particularly from third-party sources, and to use security software to protect your device from malware.

Overall, this type of attack demonstrates the potential dangers of using Zombinder and similar services to bind dropper capabilities to everyday apps. By creating a seemingly legitimate app and using Zombinder to bind a dropper, an attacker can trick victims into installing malware on their devices, allowing the attacker to gain access to sensitive information and potentially cause harm. This highlights the need for individuals to be cautious when downloading and installing apps, and for organizations to implement effective security measures to protect against such attacks.

If I were to carry out such an attack, I would do the following:

- Choose a popular game. We had made Will Hero for our Advanced Programming course project, so I already have the working code for that game.
- Inject the droppers in the code of the game using Zombinder.
- Distribute the game online on shady app stores or online through APKs.
- Create fake positive reviews
- Some unsuspecting user will download it, and voila!